

BÀI 03: SERVICE VÀ DEPENDENCY TRONG NESTJS

Mục tiêu bài 03: Bài này sẽ giới thiệu đến các bạn về các dịch vụ (Service) trong Nestjs, giới thiệu các bạn về Dependency và việc Inject lẫn nhau giữa các Service.

3.1. SERVICE TRONG NESTJS

3.1.1. Khái niệm Service

Trong **NestJS**, hai khái niệm rất quan trọng là **Service** và **Dependency**, chúng liên quan chặt chẽ với nhau, giúp xây dựng ứng dụng theo mô hình **Dependency Injection** (DI).

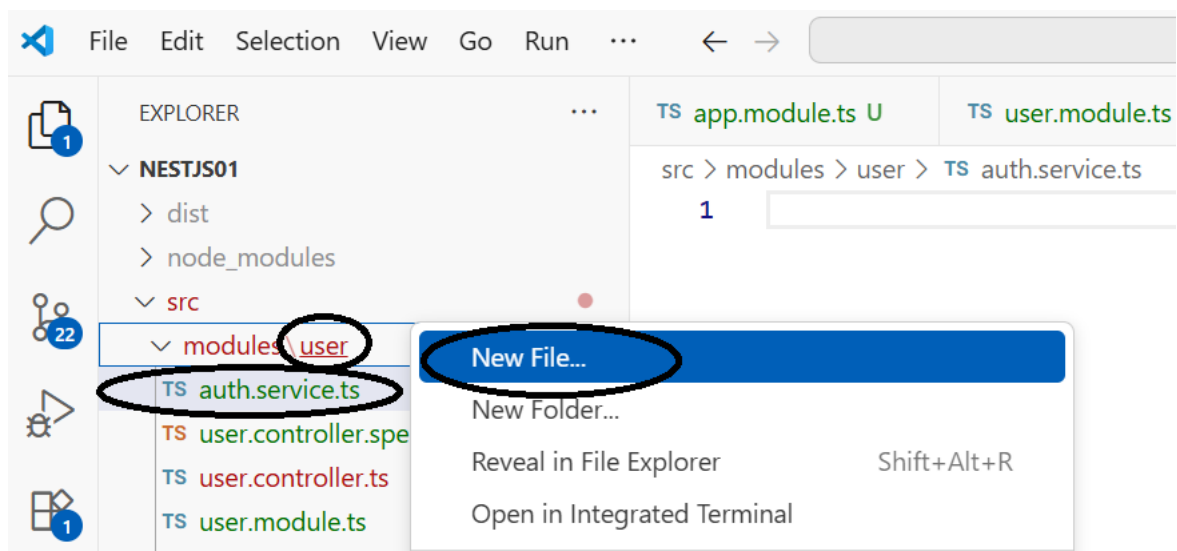
Service là nơi chứa các xử lý logic nghiệp vụ (**business logic**) của ứng dụng. Thay vì viết toàn bộ logic trong **Controller**, ta tách riêng phần xử lý vào **Service** để:

Mục đích của **Service** là tách biệt logic xử lý khỏi **Controller**, giúp code sạch hơn và dễ bảo trì hơn. Service thường được inject vào **Controller** thông qua **Dependency Injection**.

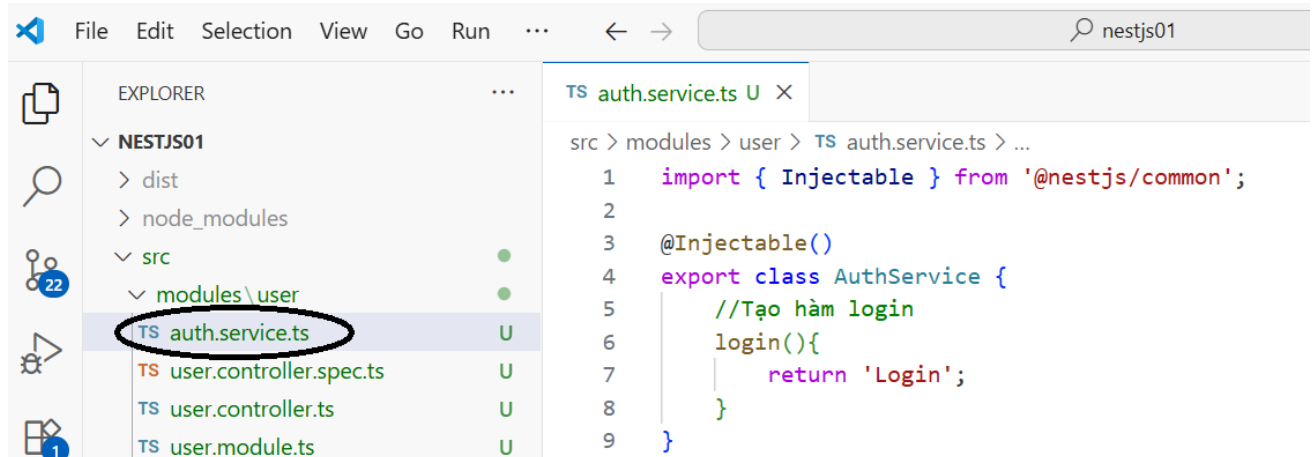
3.1.2. Tạo Service trong Nestjs

Ví dụ 3.1: Tạo một Service

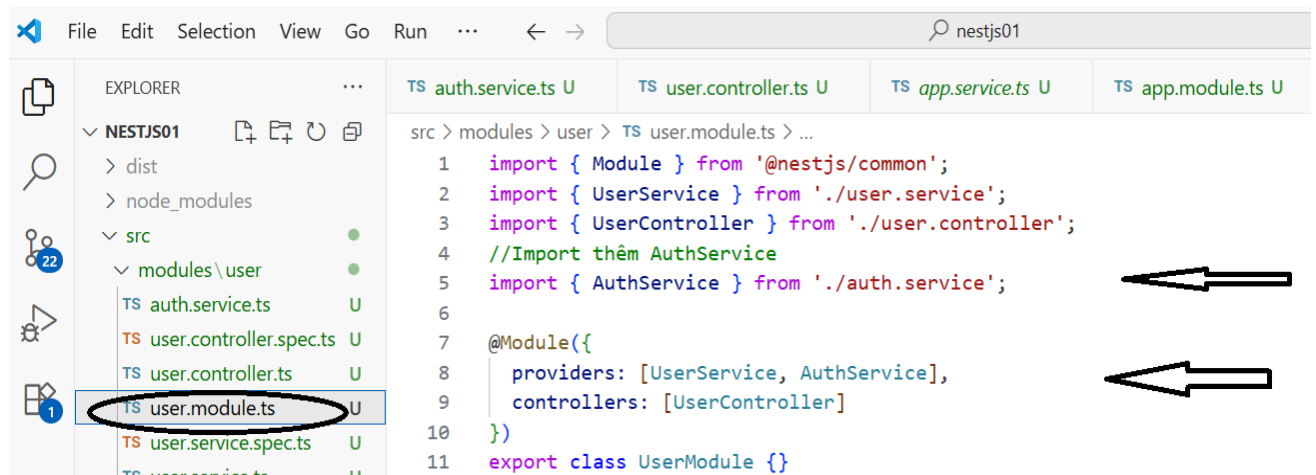
- Các bạn mở lại dự án đã tạo ở Bài 2 (*Bài trước*)
- Click chuột phải vào thư mục **user**, chọn **New File** và tạo một tập tin mới là: **auth.service.ts**



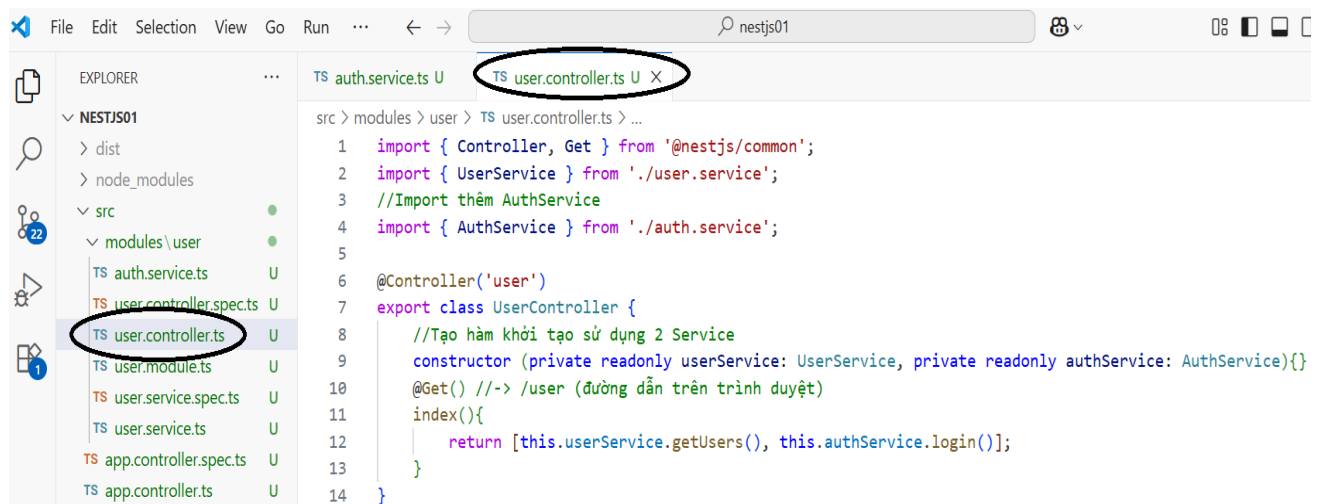
- Các bạn viết code cho tập tin: **auth.service.ts** như sau:



- Tiếp tục mở tập tin **user.module.ts** và viết code như bên dưới để cung cấp thêm Service là: **AuthService**



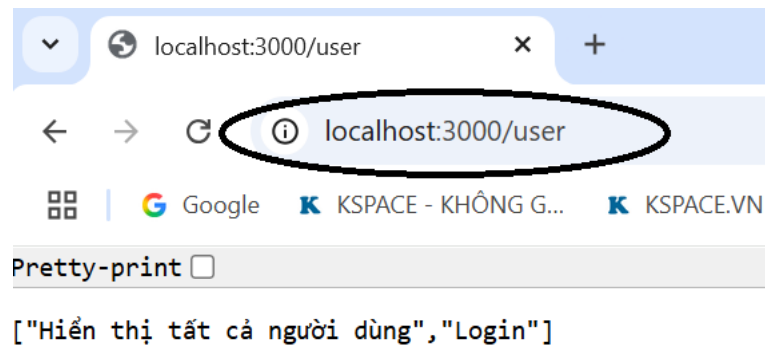
- Bây giờ tiếp tục Inject **AuthService** vừa tạo vào trong **Controller User** bằng cách: mở tập tin **user.controller.ts** và viết code như sau:



- Mở **Terminal** gõ lệnh biên dịch chương trình: **npm run start:dev**

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\AppNestjs\nestjs0 > nest start npm run start:dev
[Nest] 7964 - 07/07/2025, 3:19:41 PM LOG [NestFactory] Starting Nest application...
[Nest] 7964 - 07/07/2025, 3:19:41 PM LOG [InstanceLoader] AppModule dependencies initialized +11ms
[Nest] 7964 - 07/07/2025, 3:19:41 PM LOG [InstanceLoader] UserModule dependencies initialized +1ms
[Nest] 7964 - 07/07/2025, 3:19:41 PM LOG [RoutesResolver] AppController {/}: +5ms
[Nest] 7964 - 07/07/2025, 3:19:41 PM LOG [RouterExplorer] Mapped {/, GET} route +3ms
[Nest] 7964 - 07/07/2025, 3:19:41 PM LOG [RoutesResolver] UserController {/user}: +0ms
[Nest] 7964 - 07/07/2025, 3:19:41 PM LOG [RouterExplorer] Mapped {/user, GET} route +0ms
[Nest] 7964 - 07/07/2025, 3:19:41 PM LOG [NestApplication] Nest application successfully started +3ms
```

- Mở trình duyệt web (Chrome), gõ đường dẫn: **localhost:3000/user** sẽ cho kết quả:



- Như vậy chúng ta đã biết cách tạo thêm một **Service** và **Inject Service** vừa tạo vào chương trình rồi (*chương trình có thể thực hiện nhiều Service*)

3.2. DEPENDENCY TRONG NESTJS

3.2.1. Khái niệm Dependency

Dependency Injection (DI) là kỹ thuật mà một class nhận phụ thuộc từ bên ngoài thay vì tự tạo.

Nestjs tự động quản lý **Dependency** thông qua **IoC Container** (Inversion of Control)

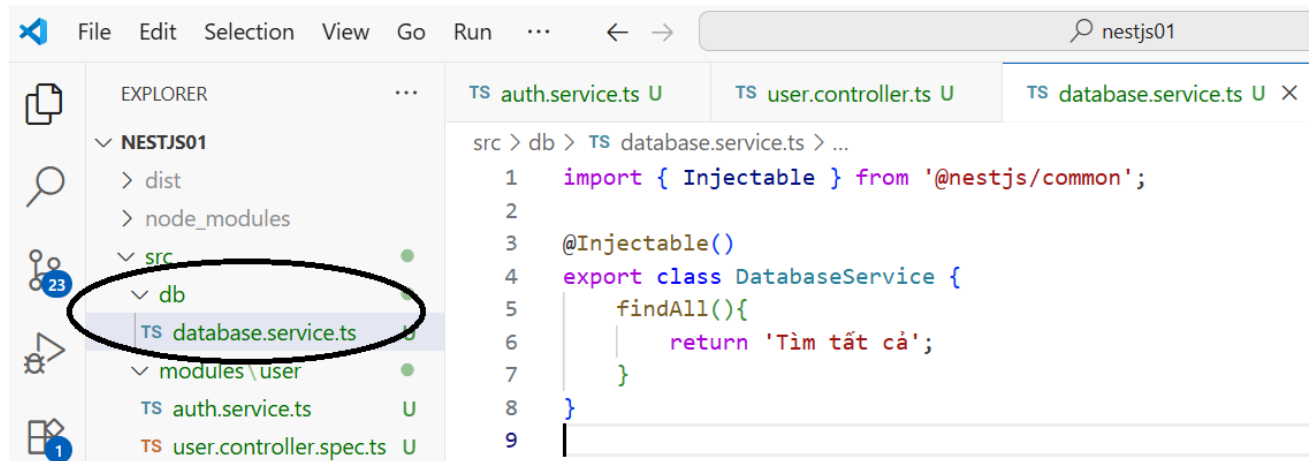
Dependency Injection giúp:

- Giảm sự phụ thuộc giữa các class
- Dễ dàng mở rộng và test code

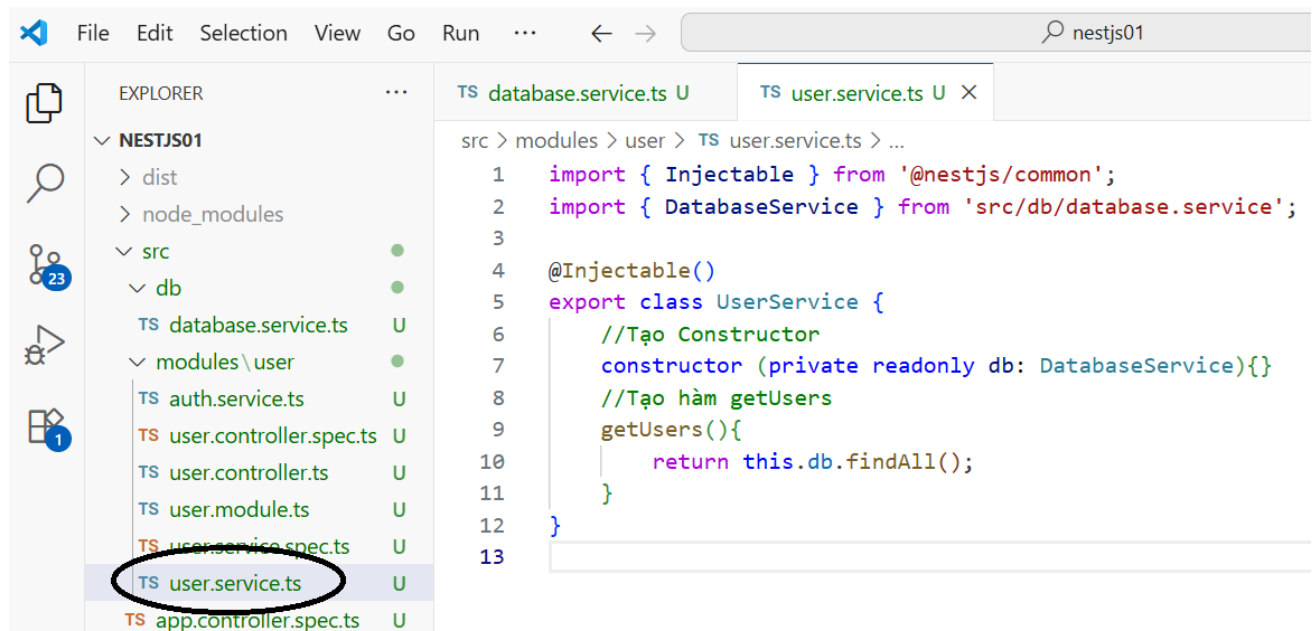
3.2.2. Tạo nhiều Service và Inject lẫn nhau

Ví dụ 3.2: Thiết lập **UserService** phụ thuộc vào **DatabaseService**

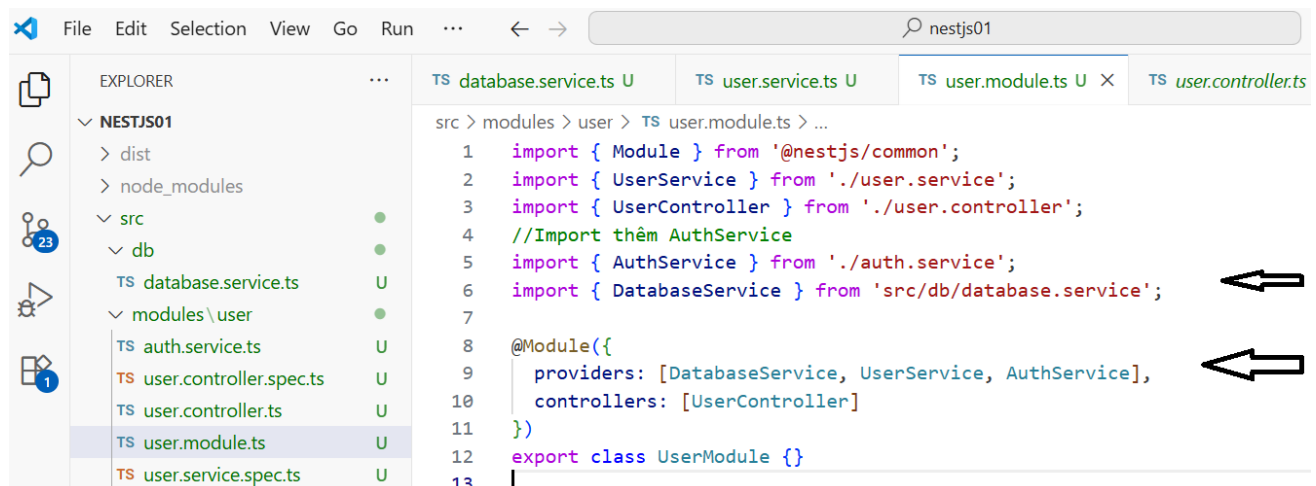
- Đầu tiên, các bạn mở lại ứng dụng Ví dụ 3.1 ở trên trong VsCode
- Tiếp tục, các bạn tạo một thư mục **db** trong thư mục **src**, trong thư mục **db** tạo một tập tin **database.service.ts** và viết code như bên dưới:



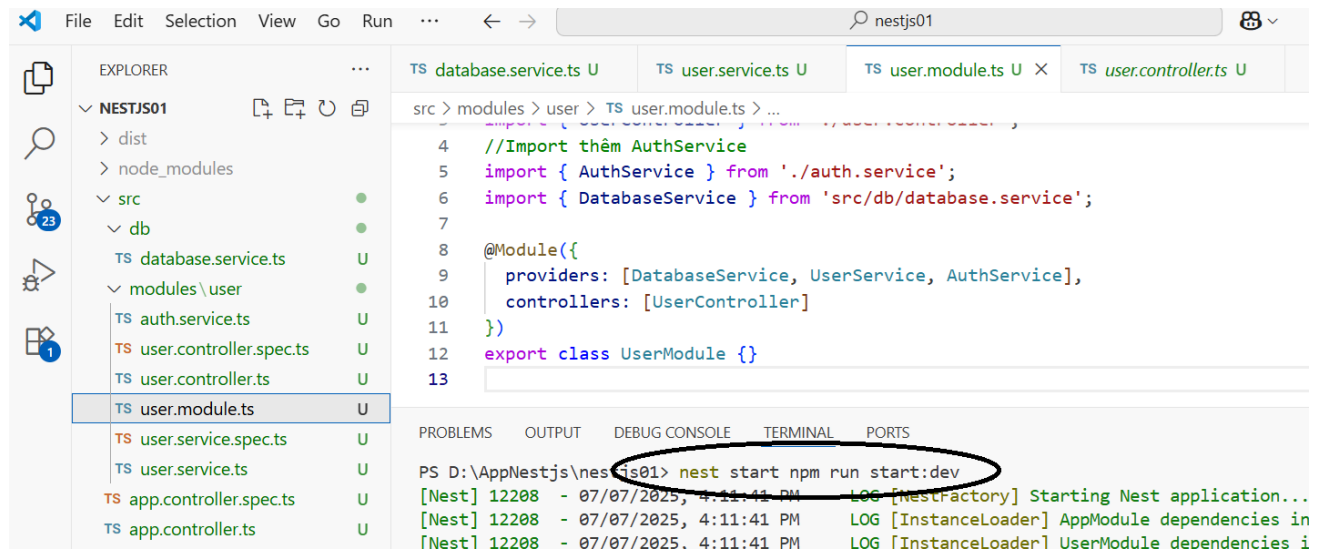
- Bây giờ, chúng ta thực hiện Inject **DatabaseService** vừa tạo vào **UserService** bằng cách mở tập tin **user.service.ts** và viết code như sau:



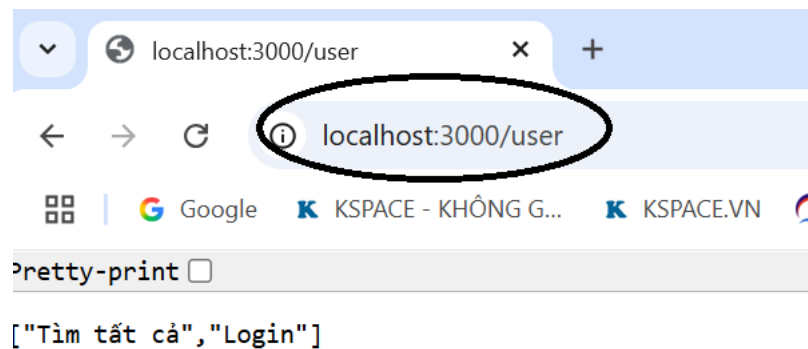
- Bây giờ, chúng ta tiếp tục gọi **DataService** vào **user.module** bằng cách mở tập tin **user.module.ts** và viết code như sau:



- Mở **Terminal** gõ lệnh biên dịch chương trình: **npm run start:dev**



- Mở trình duyệt web (Chrome), gõ đường dẫn: **localhost:3000/user** sẽ cho kết quả:



- Như vậy chúng ta thấy **UserService** phụ thuộc vào **DatabaseService** (Hàm getUsers() trong **UserService** sử dụng được hàm findAll() trong **DatabaseService**)